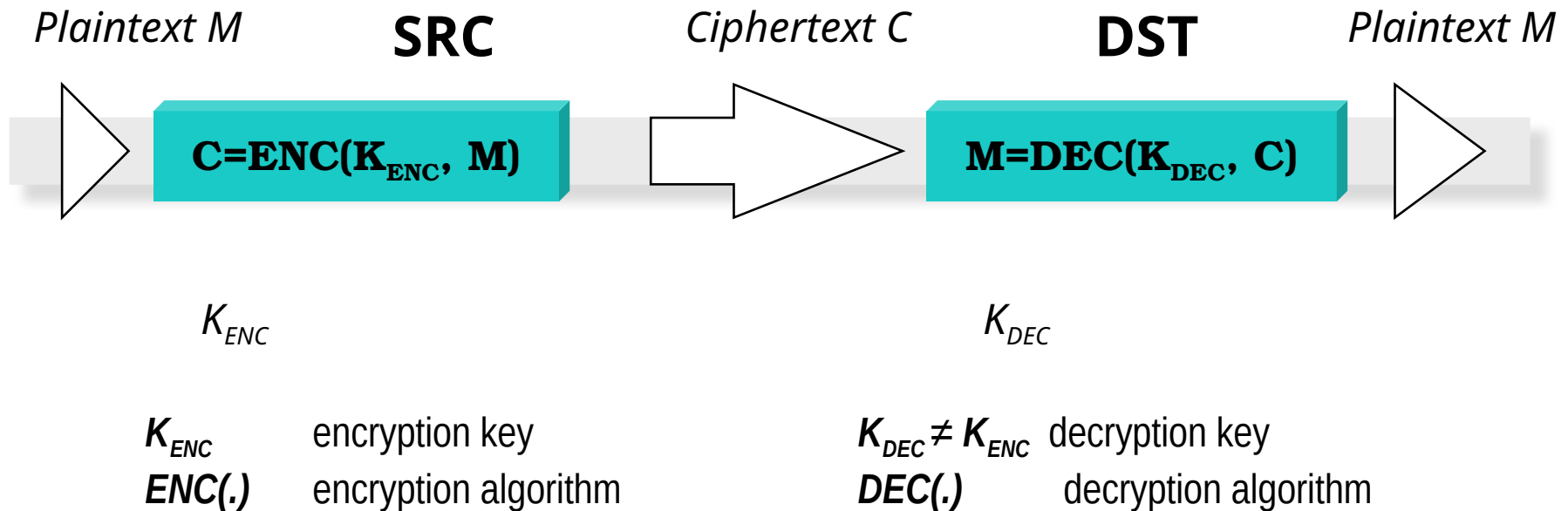


Introduction to Asymmetric Cryptography

Asymmetric cryptography

M plaintext
 C ciphertext



Encryption and decryption keys are obviously linked...
But CANNOT be determined from each other
(unless you know 'more')

Practical usage model #1: HTTPS/TLS-style

→ First phase: handshake

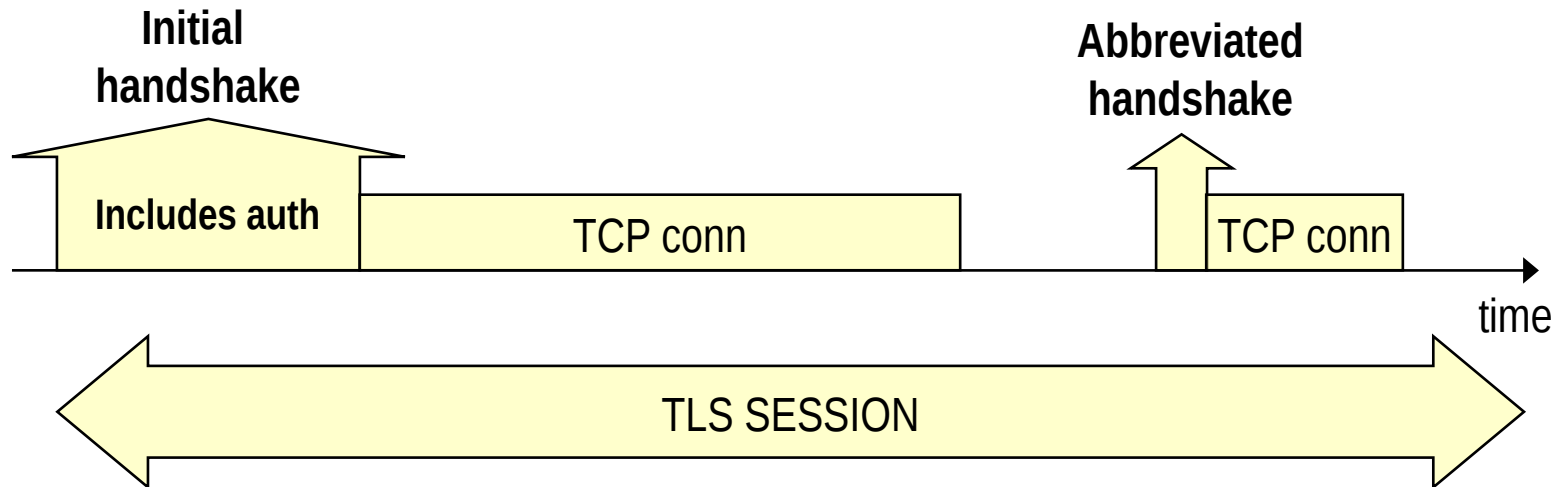
⇒ Signalling phase, sets up **shared secret** using asymmetric cryptography

→ Second phase: data transfer

⇒ Derives **symmetric encryption and message authentication keys** from shared secret derived in first phase (*use KDF, more later*)

→ Further phases: rekeying + data transfer

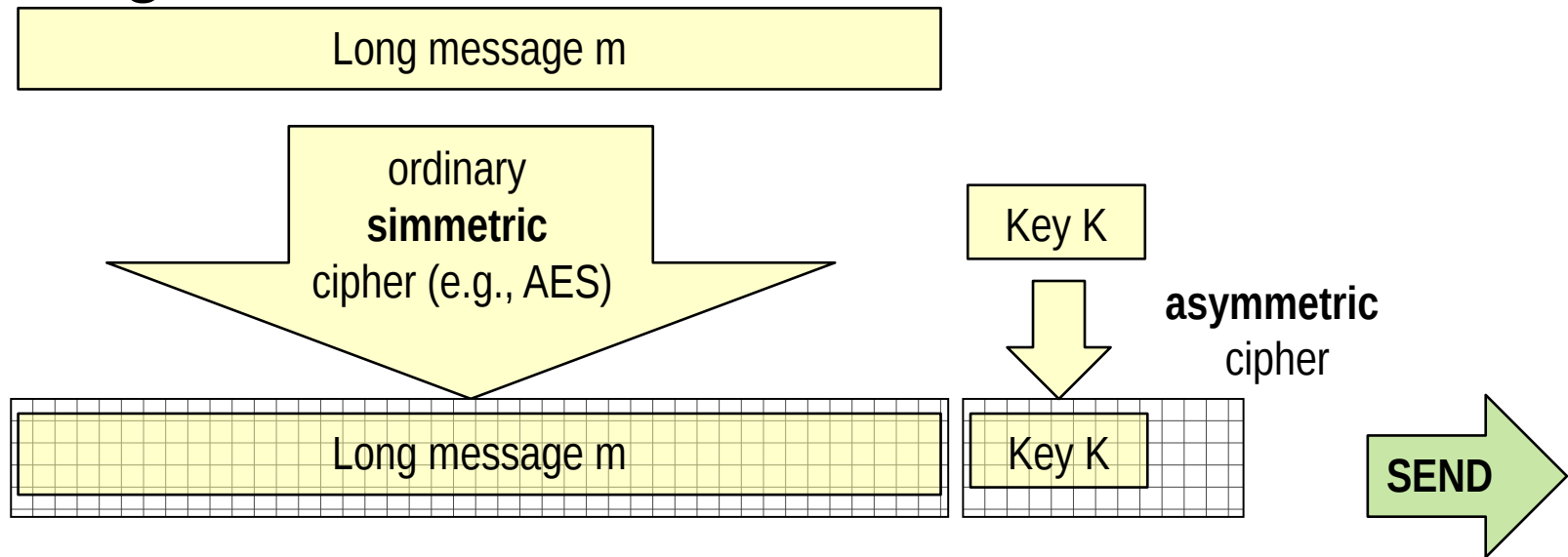
⇒ «**refresh**» symmetric encryption and message authentication **keys** still using shared secret derived in first phase (*use KDF with different nonces*)



Practical usage model #2:

Hybrid Encryption

- Generate (random) symmetric key K
- Use key K and symmetric cipher to encrypt (long) data
- Encrypt K with ASYMMETRIC cipher, and send it with message



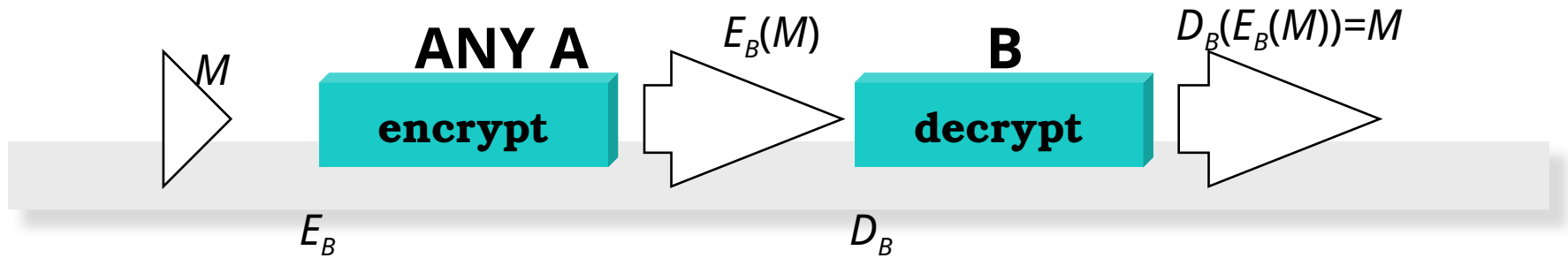
(ECIES: recently introduced in 5G subscriber Identity Concealment – see 5G SUCI)

PubKey crypto: two ways to use it

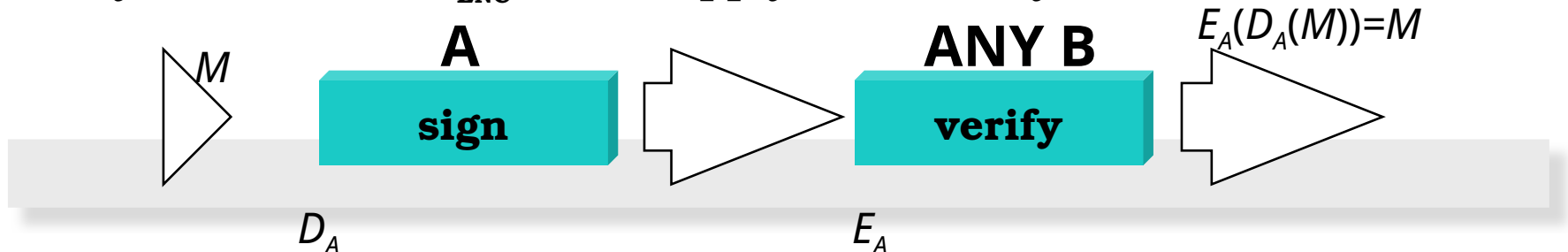
ENC and DEC are inverse operations: $\text{DEC}(\text{ENC}(M))=M$

assume commutativity: also $\text{ENC}(\text{DEC}(M))=M$

Public key encryption: anyone can encrypt M, but only B can decrypt



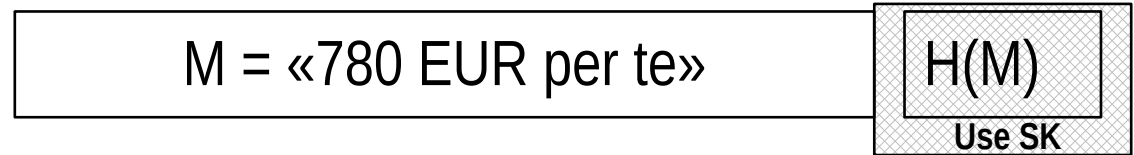
Digital signature: DUAL approach! Only A has K_{DEC} → can apply DEC (sign) but anyone else has K_{ENC} → can apply ENC (verify)!



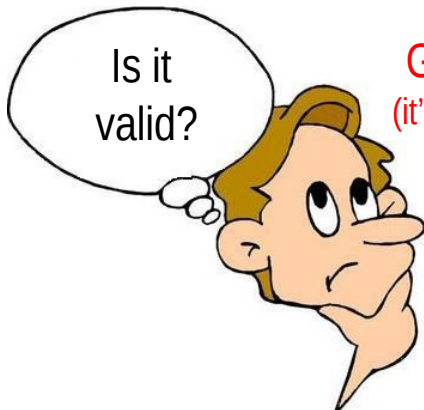
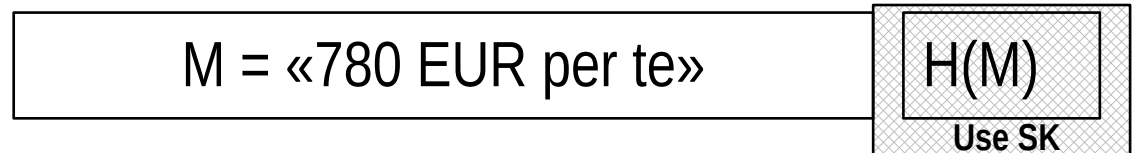
digital signature



Public Key
PK



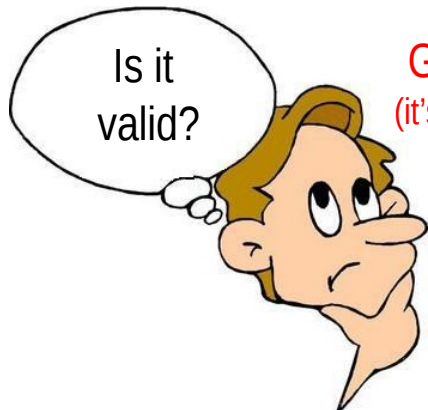
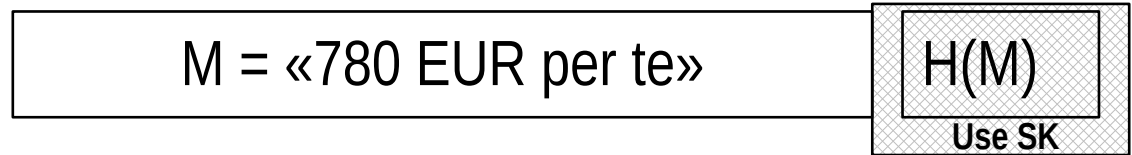
Get PK
(it's public!)



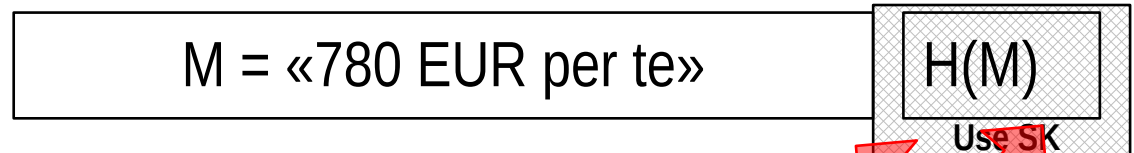
digital signature



Public Key
PK



Get PK
(it's public!)

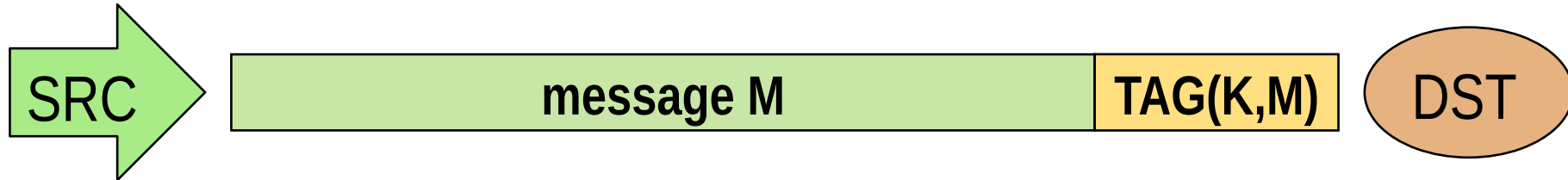


Use PK to «invert» tag

Do they match?
If Y, message is valid

Digital signature: message integrity **with** source authentication

→ Message integrity → auth tag



→ **Symmetric MAC (e.g. HMAC):**

- ⇒ K = symmetric key: shared by SRC and DST
- ⇒ DST **must know SRC key = same key K** to verify

→ **Digital Signature**

- ⇒ K = SRC private key
- ⇒ DST **just needs to know SRC PubKey** to verify!
- ⇒ **Consequence: non-repudiation a.k.a. source authentication**

Anymmetric Cryptography: Basic Algorithms

The Pioneers

IEEE TRANSACTIONS ON INFORMATION THEORY, VOL. IT-22, NO. 6, NOVEMBER 1976

New Directions in Cryptography

Invited Paper

WHITFIELD DIFFIE AND MARTIN E. HELLMAN, MEMBER,

1976: invention of asymmetric crypto

First-ever asymmetric algorithm:

Diffie-Hellman KEY AGREEMENT



A Method for Obtaining Digital Signatures and Public-Key Cryptosystems

R.L. Rivest, A. Shamir, and L. Adleman*

**1977: first ever public key
encryption & digital signature:
RSA crypto system**



Basic ingredient:

HARD (asymmetric) PROBLEM

→ Problem computationally **EASY** in one direction, but computationally **HARD** in the opposite direction

→ **Diffie-Hellman:**

⇒ DISCRETE LOGARITHM PROBLEM IN PRIME FIELDS

→ Given large prime p , g , x , **compute** $y = g^x \bmod p$ → easy

→ Given $y = g^x \bmod p$ **compute** $x = \text{DLog}_g(y) \bmod p$ → hard!!

→ **RSA:**

⇒ FACTORING A PRODUCT OF TWO LARGE PRIMES

→ Given p and q large primes, **compute** $N=p \cdot q$ → easy

→ Given N , **find factors** p, q → hard

Why modular exponentiation is easy

→ **Goal: compute $g^{1437} \bmod p$**

→ **Express x in bits**

⇒ $1437_{10} = 10110011101_2$

→ **list bits from lsb to msb**

→ **First line: initialize**

⇒ Column Square = g

⇒ Column Multiply = 1 (if $b=0$) or g (if $b=1$)

→ **Start from 2^0 lsb to msb**

⇒ For every bit

→ Square;

→ If 1: multiply to result

$b : \text{lsb} \rightarrow \text{msb}$

Square

Multiply

1

0

1

1

1

0

0

1

1

0

1

Why modular exponentiation is easy

→ **Goal: compute $g^{1437} \bmod p$**

→ **Express x in bits**

$$\Rightarrow 1437_{10} = 10110011101_2$$

→ **list bits from lsb to msb**

→ **First line: initialize**

⇒ Column Square = g

⇒ Column Multiply = 1 (if $b=0$) or g (if $b=1$)

→ **Start from 2^0 lsb to msb**

⇒ For every bit

→ Square;

→ If 1: multiply to result

→ **Complexity:**

⇒ $O(\text{nbit})$ squares

⇒ $O(\text{nbit}/2)$ multiplications

$b : \text{lsb} \rightarrow \text{msb}$

Square

Multiply

1

g

$\times g \rightarrow g$

0

g^2

.

1

$(g^2)^2 = g^4$

$\times g^4 \rightarrow g^5$

1

$(g^4)^2 = g^8$

$\times g^8 \rightarrow g^{13}$

1

$(g^8)^2 = g^{16}$

$\times g^{16} \rightarrow g^{29}$

0

$(g^{16})^2 = g^{32}$

.

0

$(g^{32})^2 = g^{64}$

.

1

$(g^{64})^2 = g^{128}$

$\times g^{128} \rightarrow g^{157}$

1

$(g^{128})^2 = g^{256}$

$\times g^{256} \rightarrow g^{413}$

0

$(g^{256})^2 = g^{512}$

.

1

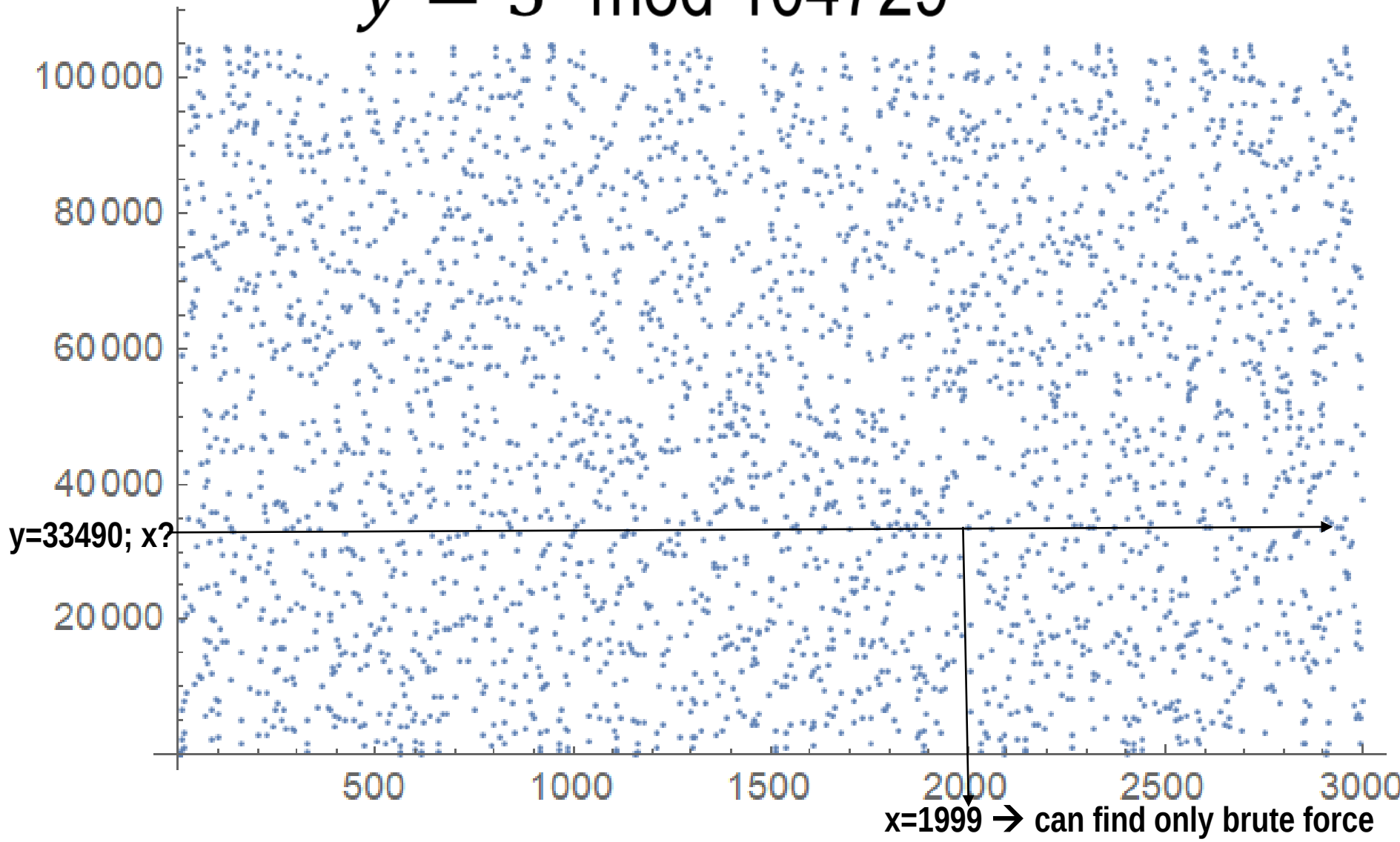
$(g^{512})^2 = g^{1024}$

$\times g^{1024} \rightarrow g^{1437}$

No algorithm such as this for the opposite problem!! That's why it is hard!

And why DLOG is hard

$$y = 3^x \bmod 104729$$



Diffie-Hellman Key Agreement

→ Diffie-Hellman protocol, 1976

- ⇒ Main crypto breakthrough in the last century
- ⇒ Invention of asymmetric cryptography
- ⇒ Though did NOT solve the PubKey cryptosystem problem
 - RSA solved it in the next year (1977)

→ What DH is: KEY AGREEMENT protocol

- ⇒ How to securely setup a shared secret (i.e., a symmetric key) at both ends, by only exchanging public values
 - i.e., WITHOUT having a secure channel

→ Hard problem used in DH: DLOG

Diffie-Hellman Key Agreement

Alice



Random
 x

SEND $g^x \bmod p$

SEND $g^y \bmod p$

COMPUTE $K = (g^y)^x \bmod p$

$K = g^{xy}$: SAME KEY
at both ends!

Bob



Random
 y

COMPUTE $K = (g^x)^y \bmod p$

$K = g^{xy}$: SAME KEY
at both ends!

Wait, I see all the
exchange!!



From $g^x, g^y \rightarrow$ hard to compute g^{xy}
Attacker must necessarily invert DLOG
for one of the two terms: computationally hard!!

DH functional limitation

→ **Does NOT implement neither a public key cryptosystem**

⇒ You cannot «transfer» data encrypted with the receiver's PK

→ **nor a digital signature!**

⇒ You cannot sign data using your SK

→ **1977: RSA to the rescue!**

The RSA algorithm

- Rivest, Shamir, Adleman, 1977
- Patented until 2000
- Widely deployed! Ultra-well known
- **HARD (asymmetric) PROBLEM USED: prime number factoring**
 - Given $N = p \times q$ (p, q very large primes)
HARD to «factor» N (= find the two values p, q)
- **Encrypt/decrypt: modular exponentiation**
 - $\text{ENC}(\text{message}, \{N, PK\}) \mod N \rightarrow \text{ciphertext} = (\text{message})^{PK}$
 - $\text{DEC}(\text{ciphertext}, \{N, SK\}) \rightarrow \text{message} = (\text{ciphertext})^{SK} \mod N$
- **Can support both encryption and digital signature**

The principle behind RSA

$m^x \bmod N$ is a periodic function

$$3^x \bmod 10 = \{3, 9, 7, 1, 3, 9, 7, 1, \dots\}$$

$$7^x \bmod 10 = \{7, 9, 3, 1, 7, 9, 3, 1, \dots\}$$

$$9^x \bmod 10 = \{9, 1, 9, 1, 9, 1, 9, 1, \dots\}$$

→ **Euler Theorem (a.k.a. Euler-Fermat Theorem, a.k.a. Euler's Totient Theorem):**

⇒ Explicit rule to compute the (maximum) period of $m^x \bmod N$

⇒ Specifically, max period = $\Phi(N)$ = **Euler's Totient function**

→ More formally:

if **m coprime with N**, i.e., $\text{GCD}(m, N) = 1$,

then **$m^{\Phi(N)} \bmod N = 1$**

Computing Euler's Totient Function

$$p \text{ prime} \rightarrow \Phi(p) = p-1$$

$$p = 7, \Phi(7) = 6 \rightarrow 3^6 \bmod 7 = 729 \bmod 7 = 1$$

$$p, q \text{ primes} \rightarrow$$

$$\Phi(p \cdot q) = \Phi(p) \cdot \Phi(q) = (p-1)(q-1) \quad \text{RSA case}$$

$$N = 10, \Phi(10) = 4 \rightarrow 3^4 \bmod 10 = 81 \bmod 10 = 1$$

$$\text{Prime } p^k \rightarrow \Phi(p^k) = \Phi(p) \cdot p^{k-1} = (p-1) p^{k-1}$$

$$N = 25, \Phi(25) = \Phi(5^2) = 4 \cdot 5$$

$$\rightarrow 3^{20} \bmod 25 = 3486784401 \bmod 25 = 1$$

$$\text{general case: } N = p_1^{k_1} \cdot \dots \cdot p_z^{k_z}$$

$$\Phi(N) = (p_1 - 1)p_1^{k_1 - 1} \cdot \dots \cdot (p_z - 1)p_z^{k_z - 1}$$

$$N = 100, \Phi(100) = \Phi(5^2 2^2) = 4 \cdot 5 \cdot 1 \cdot 2$$

$$\rightarrow 3^{40} \bmod 100 = 12157665459056928801 \bmod 100 = 1$$

Consequence of «periodicity»

$m^x \bmod N$ is a periodic function **with period $\Phi(N)$**

$$2^x \bmod 11 = \{2, 4, 8, 5, 10, 9, 7, 3, 6, 1\} \quad \Phi(11)=10$$

Compute $9 \cdot 7 \bmod 11 = ?$

Using multiplications mod 11 $\rightarrow 9 \cdot 7 \bmod 11 = 63 \bmod 11 = 8$

Alternative approach: since $9 = 2^6 \bmod 11$ and $7 = 2^7 \bmod 11$:

$$9 \cdot 7 \bmod 11 = 2^6 \cdot 2^7 \bmod 11 = 2^{6+7} \bmod 11 = 2^{13} \bmod 11 = \dots$$

$$\dots = 2^{10+3} \bmod 11 = 2^3 \bmod 11 = 2^{13 \bmod 10} \bmod 11$$

$\rightarrow$ arithmetic @ exponent = modulo $\Phi(N)$, not N !!!

$$\text{Consequence: } \begin{array}{lll} m^x = m & \bmod N & \text{if} \\ x = 1 & \bmod \Phi(N) & \end{array}$$

RSA construction

- ⇒ Generate two LARGE primes p, q (must remain SECRET!)
- ⇒ Compute *RSA module*: $N = p \times q$ (can be made public!)
- ⇒ Compute $\Phi(N) = (p - 1)(q - 1)$ (must remain SECRET!)
- ⇒ Generate public key $1 < e < \Phi(N)$ e must be coprime with $\Phi(N)$
- ⇒ Generate private key d such that $e \times d \equiv 1 \pmod{\Phi(N)}$
possible only if e invertible mod $\Phi(N) \rightarrow$ that's why e coprime with $\Phi(N)$

Security assumption: given N , must be hard to find factors p, q
→ hence hard to find $\Phi(N)$
→ an without $\Phi(N)$ hard to compute d from e

(N, e)

d

Public key

Private Key

RSA construction

(*N*, *e*)

d

Public key

Private Key

Encrypt: ENC(*M*):

$$C = M^e \bmod N$$

Decrypt: DEC(*C*):

$$\begin{aligned} M &= C^d \bmod N = \\ &= (M^e)^d \bmod N = \\ &= M^{ed} \bmod N = \\ &= M^1 \bmod N = M \end{aligned}$$

Why RSA works? Trapdoor function!

Decryption challenge: given N , e , encrypted msg m^e
compute decryption key x s.t. $(m^e)^x \bmod N = m$

Normally: HARD problem

→ exponential complexity (brute force on possible values x)

Becomes EASY when you know $\Phi(N)$

→ Equation $e \cdot x \bmod \Phi(N)$ can be solved in polynomial time

$$x = e^{-1} \bmod \Phi(N)$$

modular Inverse ← Extended Euclidean Algorithm

Reminder: Ext Euclidean algo

→ **GCD[51, 11]=1 coprime**

→ **Find a,b s.t. $51a + 11b = 1$**

a	b	val	rem
1	0	51	
0	1	11	4
1	-4	7	1
-1	5	4	1
2	-9	3	1
-3	14	1	=

Subtract 4x

Subtract 1x

Subtract 1x

Subtract 1x

$$51x(-3) + 11x(14) = 1$$

How to compute RSA inverse

→ **Public key: e**

⇒ Example: $e=13$

→ **RSA modulus: $77=11 \times 7$**

⇒ $\Phi(77)=10 \times 6=60$

⇒ $\text{GCD}(e, \Phi)=1$ (coprime, OK)

→ **Find a, b such that $\Phi a + e b = 1$**

⇒ $\text{ExtendedGCD}[60, 13]=\{1, \{5, -23\}\}$

⇒ $60 \times 5 + 13 \times (-23) = 1 \rightarrow 13 \times (-23) = 1 - 60 \times 5$

⇒ $\text{Mod}[13 \times (-23), 60] = \text{Mod}[1 - 60 \times 5, 60] = 1$

→ **Hence $13^{-1} \bmod 60 = -23 = 37$**

RSA recap: toy example

- $p = 11, q = 17$ (*secret*)
- $N = p \times q = 11 \times 17 = 187$ (*public*)
- $\Phi = (p-1) \times (q-1) = 10 \times 16 = 160$ (*secret*)

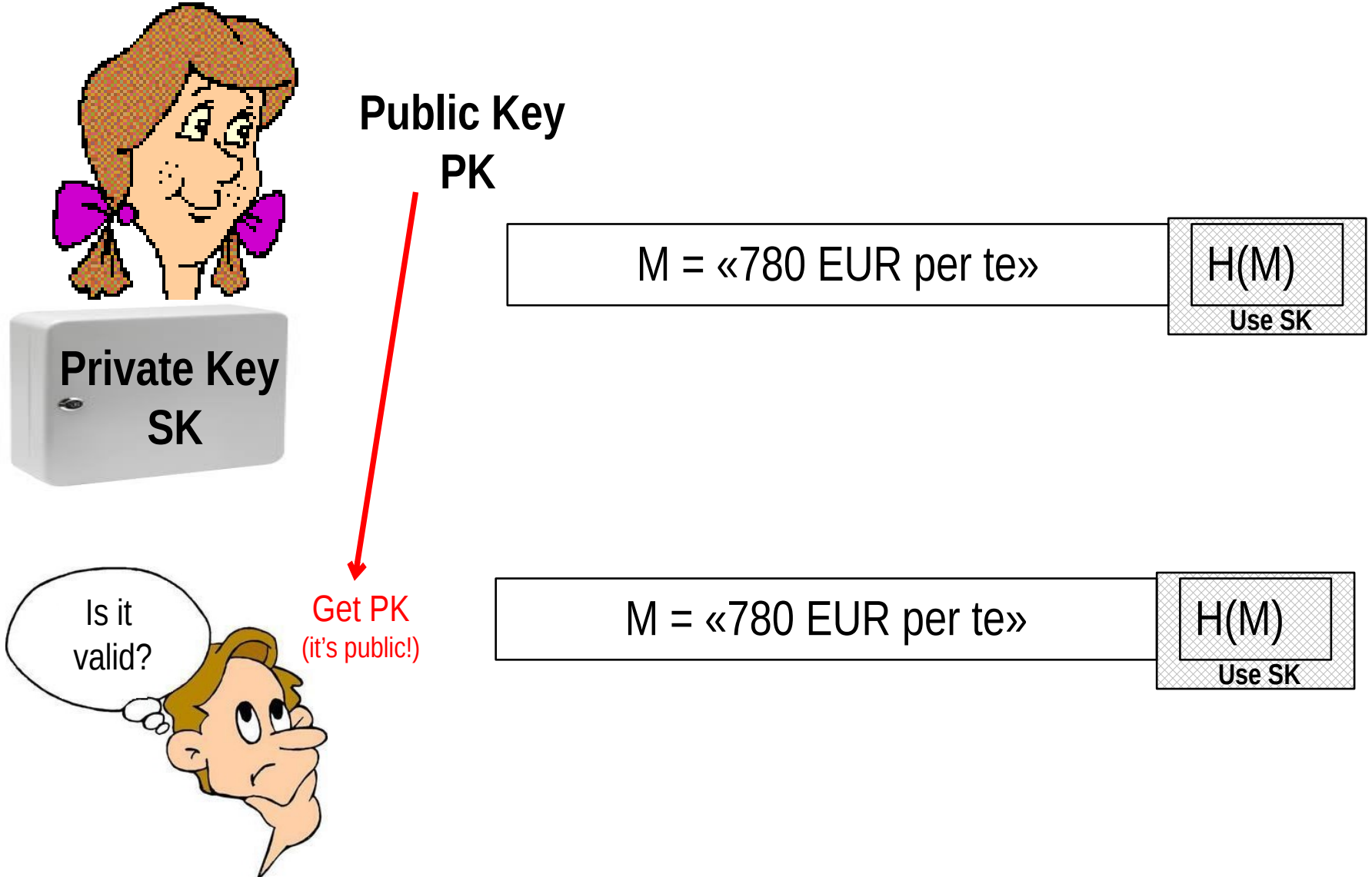
- $e = 7$ (*public, prime wrt 160*)
- $d = 7^{-1} \bmod 160 = 23$ (*secret*)
 - ⇒ Note: $23 \times 7 \equiv 161 = 160 + 1 = 1 \pmod{160}$

- **Public Key Encryption: $C = M^7 \bmod 187$**
 - ⇒ Decrypt: $(M^7)^{23} \bmod 187 = M^{7 \times 23} \bmod 187 = M$

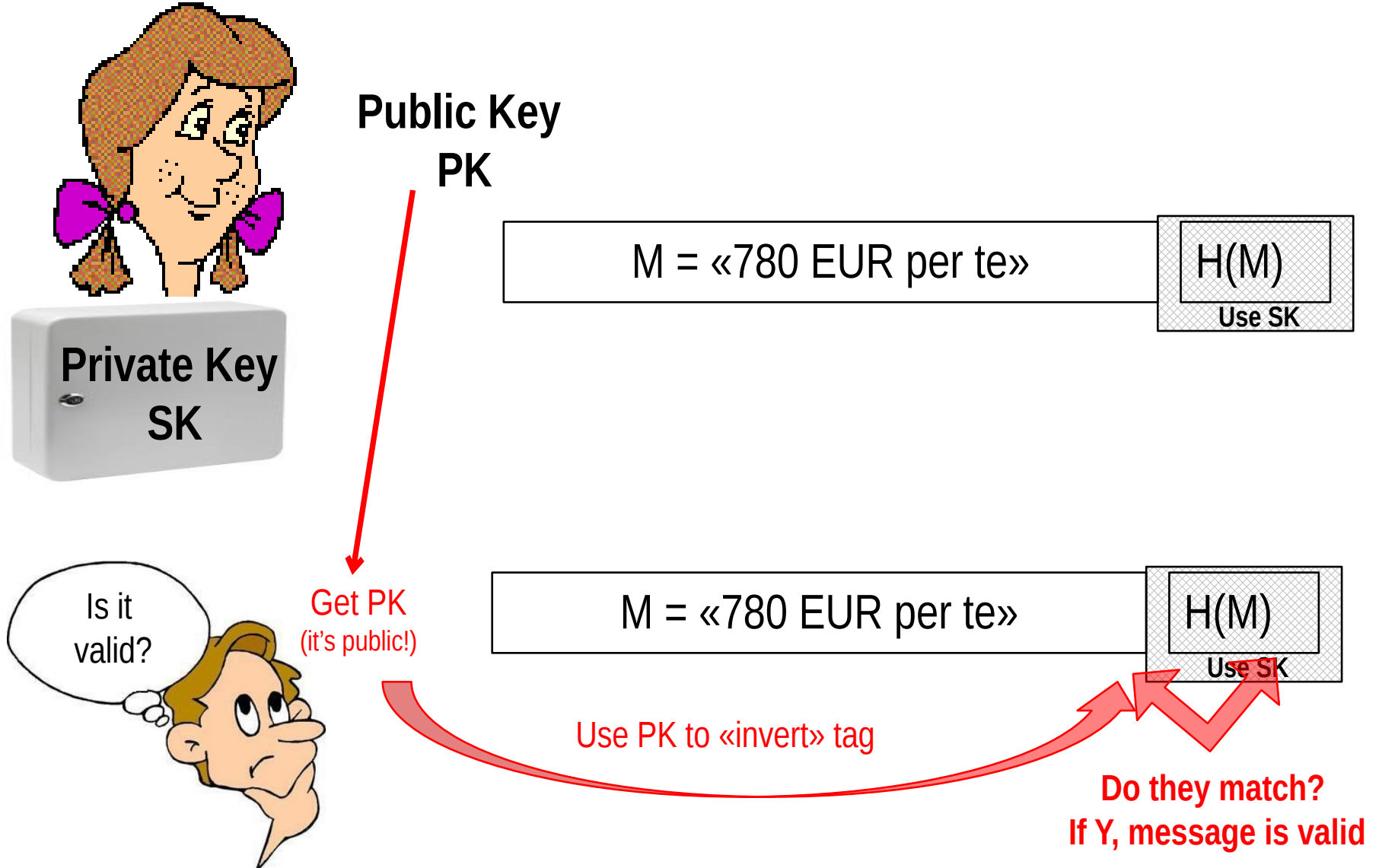
- **Digital Signature: send M , $TAG = H(M)^{23} \bmod 187$**
 - ⇒ Verify sign: $TAG^7 \bmod 187 = (H(M)^{23})^7 \bmod 187 = H(M)$
 - ⇒ check hash matches with message M provided

Digital Certificates and Public Key Infrastructure

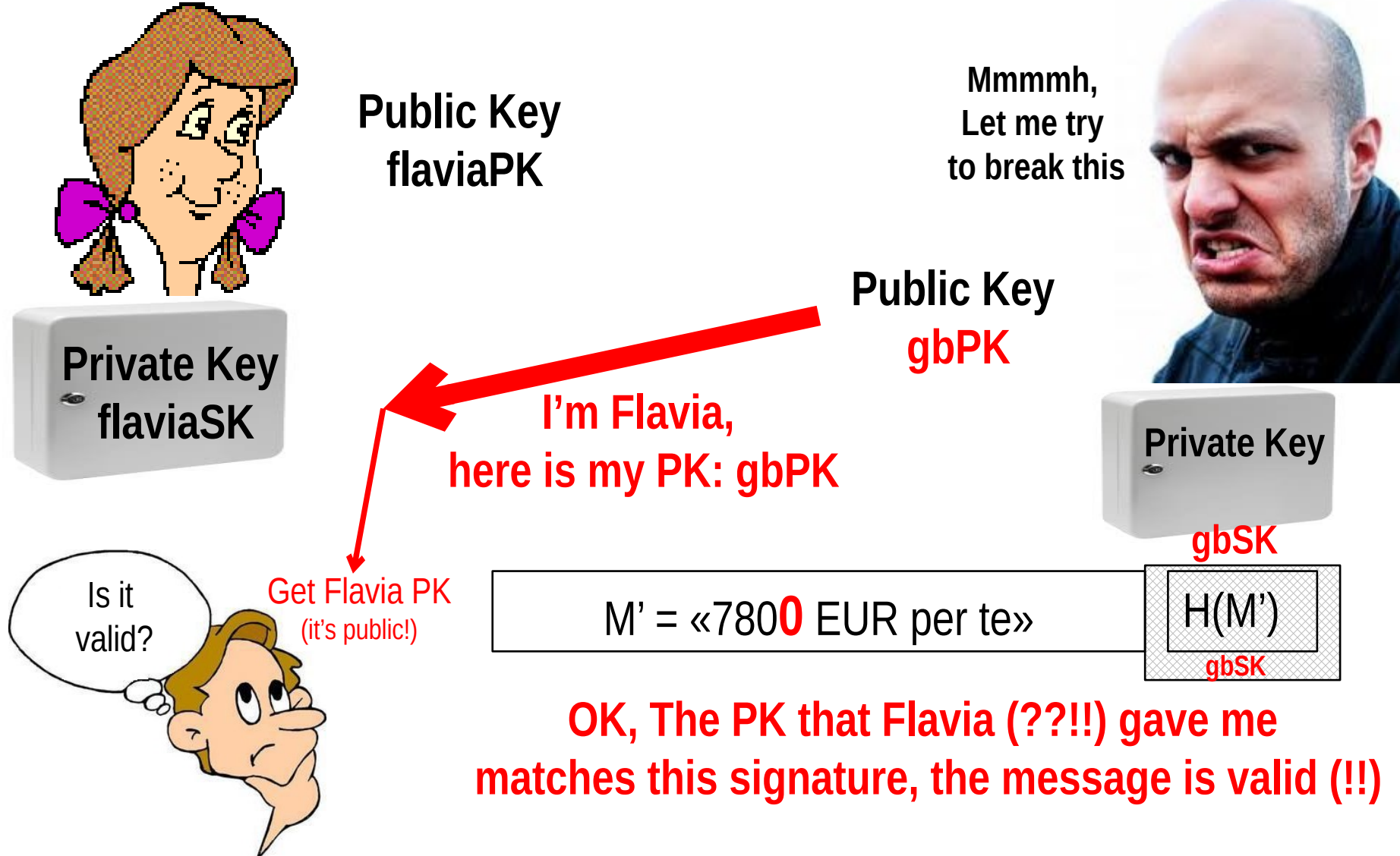
digital signature: review



digital signature: review

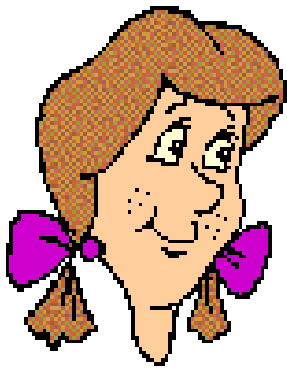


digital signature: problem



RSA Key Transport, review

Flavia wants to access her bank account
using a secure communication



Hello dear, would you mind sending me your PK?

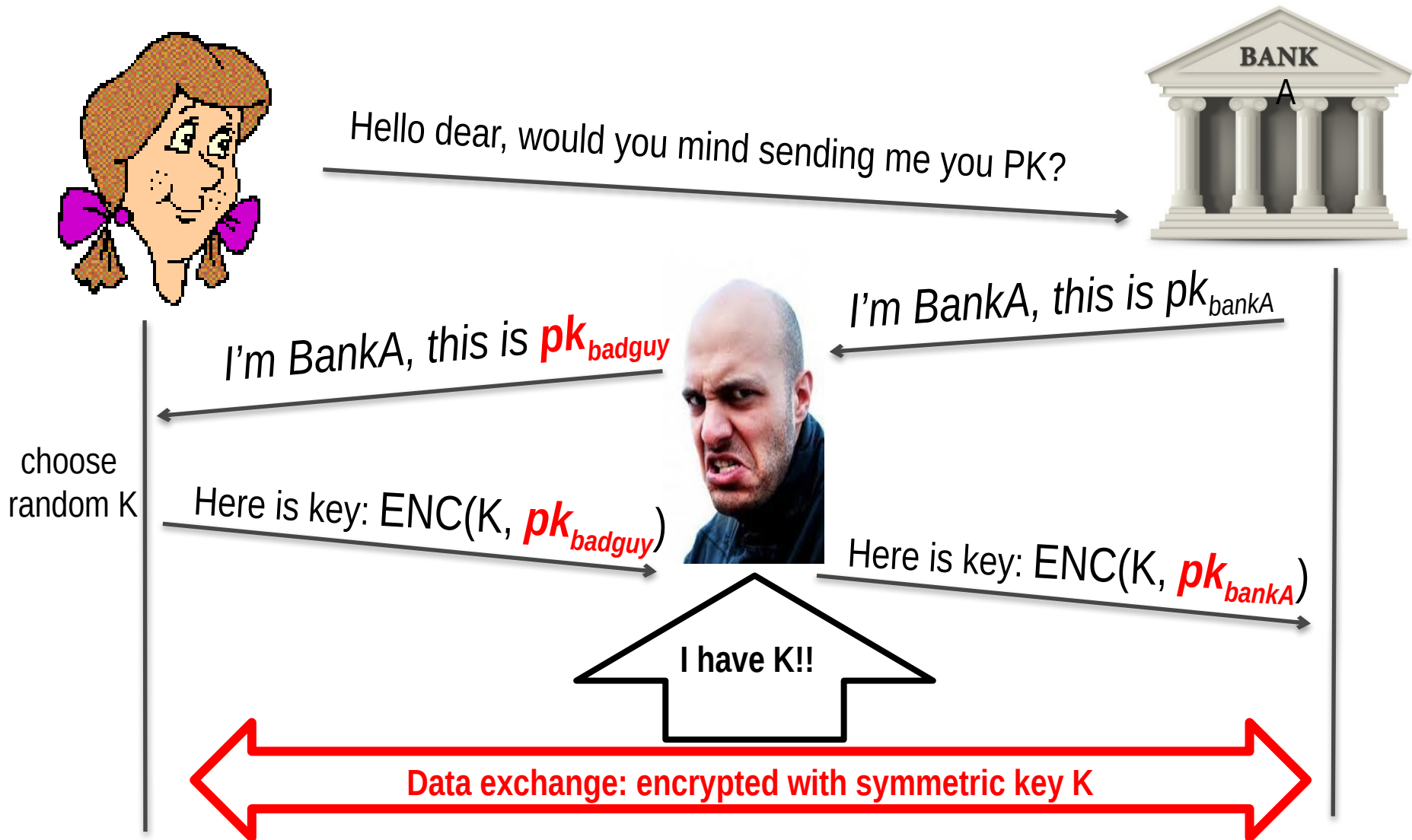
Here you are: $BankA \rightarrow pk_{bankA}$

choose
random K

Here is the secret key we'll use in our session: $ENC(K, pk_{BankA})$

Data exchange: encrypted with symmetric key K

RSA Key Transport, MITM attack



What about DH? MITM, again!!

Alice

Eve!

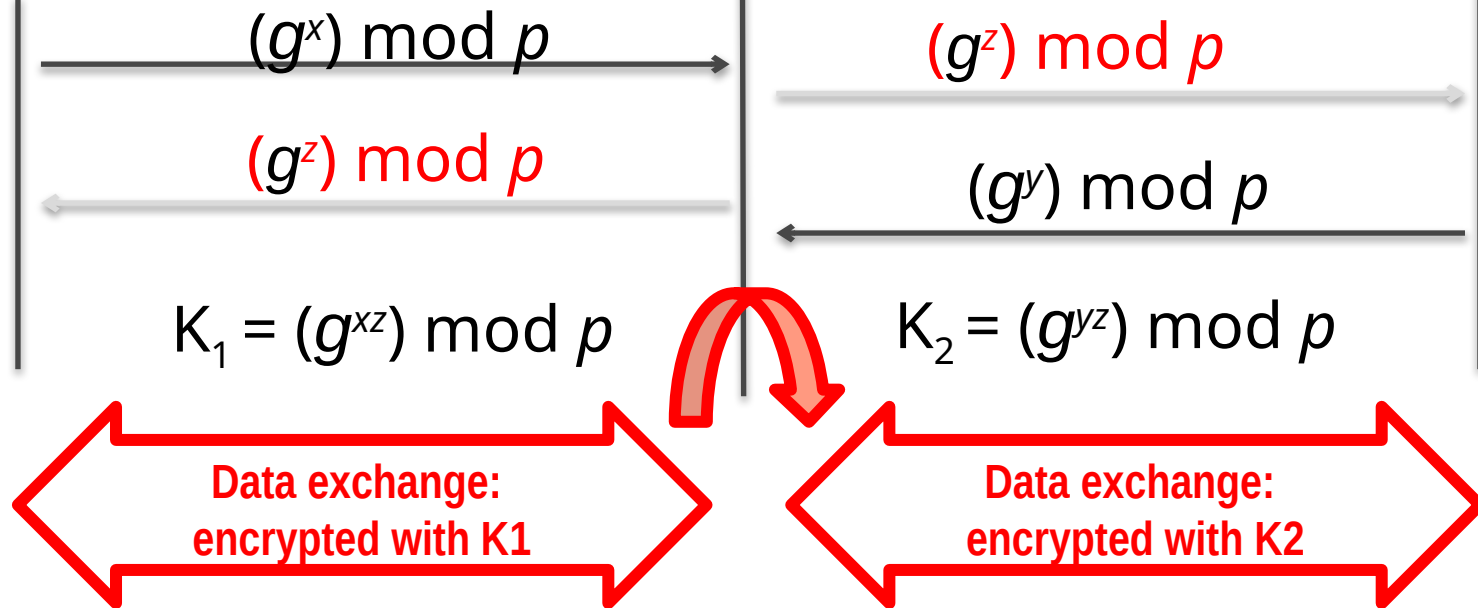
Bob



choose random
 $x \in \{0,1\}^s$

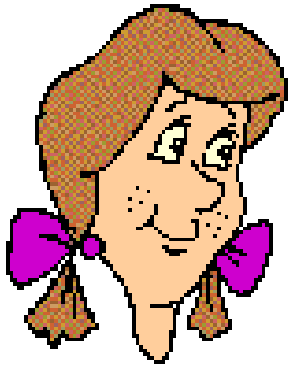


choose random
 $y \in \{0,1\}^s$



Three different scenarios, SAME problem!!

We need a way to 'cryptographically bind' a public key to an identity!



I'm Flavia, my public key is 876543



I'm Flavia, my public key is 876543



NO
YOU'RE NOT,
I CAN
SPOT YOU

The solution: Digital Certificates

Main Role of a digital certificate

→ To Bind a public key to a subject

⇒ A person, a company, a legal entity, ...

Subject NAME

Subject Public Key



→ Cryptographic binding

⇒ Digital signature by a TRUSTED THIRD PARTY: a “certification Authority” that guarantees integrity of the binding

Why we need certificates?

➔ **User X knows ALL public keys of all remaining users**

⇒ No need for certificates (direct trust model)

⇒ BUT: unviable, does not scale

➔ **User X trusts TTP, who guarantees for other users**

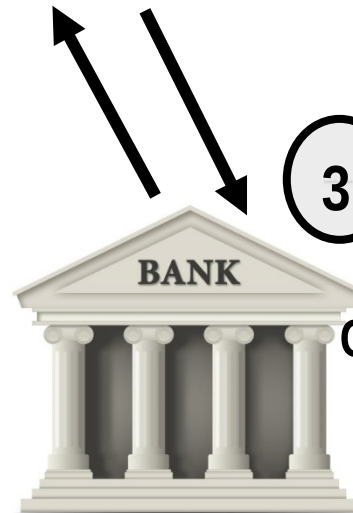
⇒ Public Key Infrastructure

A) Issuing a Certificate (offline)

CERTIFICATION
AUTHORITY

4

$CERT = (BankName, BankPK)_{CA_sign}$



3 Here is my BankName
and here my PK
(via secure channel)

$CERT = (BankName, BankPK)_{CA_sign}$

SK

1

Generate PK and SK pair

2

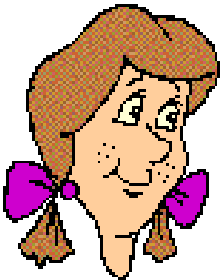
Store SK locally
(never disclosed or TXed!)

B) Verifying Certificate validity

CERTIFICATION
AUTHORITY

1

Here I am: name and PK
(BankName, BankPK)_{CA_sign}



CERT = (BankName, BankPK)_{CA_sign}

SK

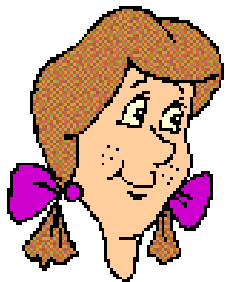
2 Do I trust the CA you have chosen?
(Is in my list of trusted ones?) → OK

3 Is the CA signature correct? → OK

DOES THIS MEAN THAT
I'M REALLY TALKING TO THE BANK?

B) Verifying Certificate validity

CERTIFICATION
AUTHORITY



Here I am: name and PK
(BankName, BankPK)_{CA_sign}



CERT = (BankName, BankPK)_{CA_sign}

3

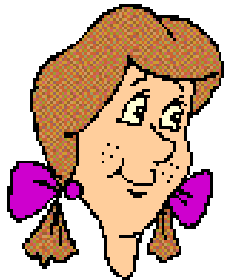
Is the CA signature correct? → OK

DOES THIS MEAN THAT
I'M REALLY TALKING TO THE BANK?

NOT YET – what about cert replay?!

B) Verifying Certificate validity

CERTIFICATION
AUTHORITY



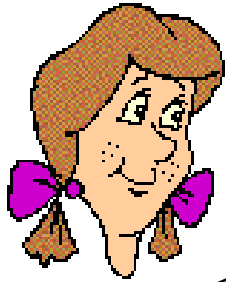
TWO WAYS TO DO THIS:

- 1) Ask Bank to sign something fresh
- 2) Ask bank to decrypt something fresh

4

Let's «ask» the bank to
PROVE possession of the
private key SK associated to the cert!

Proof of knowledge of SK via digital signature



Generate
fresh
NONCE
(challenge)

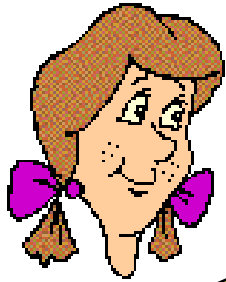
Certificate(BankName, BankPK)_{CA_sign}

Prove knowledge of SK by signing a fresh NONCE

(NONCE)_{Bank_sign}

If bank signature «opens»
with certificate PK,
then Bank is authenticated!

Proof of knowledge of SK via PubKey encryption



Generate
fresh
NONCE
(challenge)

Certificate(BankName, BankPK)_{CA_sign}

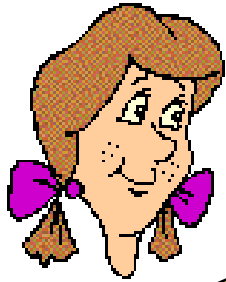
Prove you know SK by decrypting this: ENC (NONCE)_{Bank_PK}

Here you are: NONCE

If bank returns «decrypted» NONCE
then Bank is authenticated!

Practical TLS approach

(when RSA Key Transport used)



Generate
fresh
Key K

Certificate(BankName, BankPK)_{CA_sign}

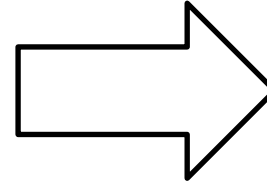
ENC (key K)_{Bank_PK}

Data exchange: encrypted with symmetric key K

Bank can communicate only
if it is Authentic!! If Rogue/MITM,
cannot decrypt key K

Certificate chains

UNIVERSETRUST



LUNATRUST

LUNATRUST? Not in my list of trusted.

Who «certifies» LUNATRUST?

UNIVERSETRUST: OK, this is in my list, let's go on!



CERTIFICATE(BankName,PK)_{LUNATRUST}



Very brief intro to PKI

Public Key Infrastructure

Public Key Infrastructure (PKI)

- ➔ **A PKI specifies protocols, policies, and technical mechanisms needed to support exchange of public keys**
- ➔ **A PKI architecture requires:**
 - ⇒ Standard format for certificates
 - ⇒ Relation among CAs, and with end users
 - ⇒ Policies for issuing and REVOKING (!!) certificates
 - ⇒ *Directory services*
- ➔ **Typical certificate format: X.509**

X.509 certificates (high level)

Standard version, Temporal validity, Unique serial number, Algorithms, etc
CA identity
User identity
Public key of user
Digital signature by the CA

X.509 certificates: a real example



```
marlon@ubuntu:~/Desktop$ openssl x509 -in www.facebook.com
-----BEGIN CERTIFICATE-----
MIIGMjCCBRqgAwIBAgIQDG/IWVf6H1/JZyyf5lzb5jANBgkqhkiG9w0BAQUFADBm
MQswCQYDVQQGEwJVUzEVMBMGA1UEChMMRGlnaUNlcnQgSW5jMRkwFwYDVQQLEwB3
d3cuZGlnaWNaW1cnQuY29tMSUwIiwYDVQQDEwxEaWdpQ2VydCBIEFZzc3VyYW5j
ZSBDO30zMB4XDTEwMTEwMDAwMFoXDTEzMTIwMjI0VjEwZjE1MAkGA1UEBhMCVVMx
EzARBgNVBAGTCkNhbmG1mb3JuaWEuXzEjAQBgNVBACTCVBhbg8gQWw0bzEX
MBUGA1UEChMORMmFjZWJvb2ssIEluYy4xGTAXBGNVBAMTEHd3dy5mYWNlYm9vay5j
b20wZzZ8wDQYJKoZIhvcNAQEBBQADgYOAAMIGJAoGBAMHffWNBvcTk+mUzE3jVYjw
p2HzsZa/I466h6ftB/neLeuox7ytd6ZeJQMDNuN99Dxq2byty4zFr4mD11BFn//
twCe+g6ZFwXSGtcKxq375AcIP9sEplZppe3Wh7aIXYP16Maz/8A0H52jhXDtonYU
e3A+77BCCzjWggAj3WN1AgMBAAGjggNaMIIDVjAFBgNVHSMEGDAWgBRQ6nOJ2yn7
EI+e5QEg1N55mUId9zAdBgNVHQ4EFgQUqldKM7bs1W6BE6Y2LvR7Q1jzj0QwKQYD
VR0BCIwIIQd3d3LmZhY2Vib29rLmNvbYIMZmFjZWJvb2suY29tMHsGCCsGAQUF
BwEBBG8wbTAKBggrBgEFBQcwAYYYaHR0cDovL29jc3AuZGlnaWNaW1cnQuY29tMEUG
CCsGAQFUBzACHjlodHRwOi8vY2FjZXJ0cy5kaWdpY2VydC5jb20vRGlnaUNlcnRI
aWdoQXNzdXJhbmNlQ0EtMy5jcncwDgYDVROPAQH/BAQDAgWgMAWGA1UdEwEB/wQC
MAAwZQYDVROfBF4wXDAsocGgKIYmaHR0cDovL2NybdMuZGlnaWNaW1cnQuY29tL2Nh
My0yMDEwaS5jcmwwLKAQoCiGJmH0dHA6Ly9jcmmw0LmRpZ2ljZXJ0LmNvbS9jYTMt
MjAxMGkuY3J5SjIIBXgYDVROgBIIbVtCCABkwggG1BgtghkgBhv1sAQMAATCCAaQw
OgYIKwYBBQUHAgEwLmh0dHA6Ly93d3cuZGlnaWNaW1cnQuY29tL3NzbC1jcmHtcmVw
b3NpdG9yeS5odG0wggFkBggrBgEFBQcCAjCCAVYeggsFAEEAbgB5ACAAdQBzAGUA
IABvAGYAIABoAGgAaQwBzACAAQwB1AHIAIAdABpAGYAaQwBjAGEAdAB1ACAAyWbVAG4A
cwBOAGkAdAB1AHQAZQBzACAAyQBjAGMAZQBwAHQAYQBuAGMAZQAAG8AZgAgAHQA
aAB1ACAARABpAGcAaQBDAGUAcgB0ACAAQwBQAC8AQwBQAFMAIAbhAG4AZAAGAHQA
aAB1ACAAGUgB1AGwAeQwBAG4AZwAgAFAYQByAHQAeQAgAEEAZwByAGUAZQBtAGUA
bgB0ACAAAdwBoAGkAYWBoACAAbABpAG0AaQBOACAAAbABpAGEAYgBpAGwAaQBOAHKA
IABhAG4AZAAGAGEAcgB1ACAAaQwBAGMAAbwBYAHAAAbwB1AGQAIAB0AGUA
cgB1AGkAbgAgAGIAeQAgAHIAZQBmAGUAcgB1AG4AYwB1AC4wHQYDVROlBBYwFAYI
KwYBBQUHAgEGCCsGAQUFBwMCMCAOGCSqGSIb3DQEBBQUAAIIBAQAlM16QP60C/t6S
0p4S9+8Wao26ZqBarmZ2vEoSE+OS1vcP1LwB1SD08P2sZt4kGK/uor9fo+xectYg
GtLGjwcNev+lj30HF06ZgQBjYCNjcAAFsUd2AY39+wD6KK0QFyVdQwUAdF1p1aY
8DggH3cVeau14wQKd8nDtZ1Xdk80bncaYTDvmrptUT9RPPXAtMQgl+kmEODDGERB
2Sb30UvyyoaTDTQXFvUj1hcspgGHW14e6yCX+hXG70mZjUkkLHWqAYkM8J/w8Khwu
gqeCEJjrS1oyfLGPXDKAx9xtb3+v2DdAE0j8xCWg/hvleSrYh1SBXmU1zHyHHVE
yieOb6nD
-----END CERTIFICATE-----
```

Giuseppe Dianzani

Certificate Viewer: www.facebook.com

General Details

This certificate has been verified for the following usages:
SSL Server Certificate

Issued To

Common Name (CN)	www.facebook.com
Organization (O)	Facebook, Inc.
Organizational Unit (OU)	<Not Part Of Certificate>
Serial Number	0C:6F:C8:59:57:FA:1F:5F:C9:67:2C:9F:E6:5C:DB:E6

Issued By

Common Name (CN)	DigiCert High Assurance CA-3
Organization (O)	DigiCert Inc
Organizational Unit (OU)	www.digicert.com

Validity Period

Issued On	11/15/10
Expires On	12/3/13

Fingerprints

SHA-256 Fingerprint	BB A9 12 B4 FE 2F 26 88 7D 79 0B C4 2F 7A 98 7B C8 D8 1C 21 B1 90 C4 46 5B C3 1A 2C 5B 6F D2 31
SHA-1 Fingerprint	63 08 84 E2 79 CB 11 07 F1 FB 8A 6B 11 A6 4D 1B 14 76 3F 8E

Close

X.509 certificates: a real example

Version: 3 (0x2)

Serial Number:

0c:6f:c8:59:57:fa:1f:5f:c9:67:2c:9f:e6:5c:db:e6

Signature Algorithm: sha1WithRSAEncryption

Issuer: C=US, O=DigiCert Inc, OU=www.digicert.com, CN=DigiCert High Assurance CA-3

Validity

Not Before: Nov 15 00:00:00 2010 GMT

Not After : Dec 2 23:59:59 2013 GMT

Subject: C=US, ST=California, L=Palo Alto, O=Facebook, Inc., CN=www.facebook.com

Subject Public Key Info:

Public Key Algorithm: rsaEncryption

RSA Public Key: (1024 bit)

Modulus (1024 bit):

00:c1:df:7d:63:41:bd:c4:e4:fa:65:33:13:78:d5: (... deleted ...) 0b:38:d6:82:00:23:dd:63:75

Exponent: 65537 (0x10001)

X509v3 extensions: (deleted)

X509v3 Subject Key Identifier:

AA:57:4A:33:B6:EC:D5:6E:81:13:A6:36:5E:F4:7B:43:58:F3:8F:44

X509v3 Subject Alternative Name:

DNS:www.facebook.com, DNS:facebook.com

X509v3 Key Usage: critical

Digital Signature, Key Encipherment

X509v3 Basic Constraints: critical

CA:FALSE

X509v3 Extended Key Usage:

TLS Web Server Authentication, TLS Web Client Authentication

Signature Algorithm: sha1WithRSAEncryption

25:33:5e:90:3f:ad:02:fe:de:92:d2:9e:12:f7:ef:16:6a:8d: (... deleted ...) 8e:6f:a9:c3

Public Key Cryptography Standards

→ PKCS family

- ⇒ PKCS #1 -- #15
- ⇒ Some obsoleted

→ Standardize many PKI related aspects (and more)

- ⇒ PKCS#1: RSA
- ⇒ PKCS#3: DH
- ⇒ PKCS#13: ECC
- ⇒ PKCS#6: X.509 extensions
- ⇒ PKCS 10: Certification Request Syntax Standard
- ⇒ PKCS 11, 15: Cryptographic Token Interface Standard - API for signing and verifying data by a device that holds the key
- ⇒ PKCS 12: Personal Information Exchange Syntax Standard - file format for storing certificate and private key - used to move private information between browsers

Example: Certificate Signing Request

- **A certificate signing request (also CSR or certification request) is a message sent from an applicant to a certificate authority in order to apply for a digital identity certificate**
- **The most common format for CSRs is the PKCS#10 specification**
- **Operations:**
 - ⇒ the applicant first generates a key pair, keeping the private key secret
 - ⇒ the applicant generates a CSR contains information identifying herself (X.509 subject field), optional X.509 extensions (e.g. key usage: RSA authentication for web servers) and the public key chosen by the applicant
 - ⇒ The CSR may be accompanied by other credentials or proofs of identity required by the certificate authority, and the certificate authority may contact the applicant for further information

X509v3 extensions

- An X.509 v3 certificate contains an extension field that permits any number of additional fields to be added to the certificate**
- Certificate extensions provide a way of adding information such as alternative subject names and usage restrictions to certificates**

Some standard extensions

→ **Authority Key Identifier**

- ⇒ The authority key identifier extension provides a means of identifying the public key corresponding to the private key used to sign a certificate

→ **Subject Key Identifier**

- ⇒ The subject key identifier extension provides a means of identifying certificates that contain a particular public key

→ **Key Usage**

- ⇒ The key usage extension defines the purpose (e.g., encipherment, signature, certificate signing) of the key contained in the certificate.
- ⇒ digitalSignature, nonRepudiation, contentCommitment, keyEncipherment, dataEncipherment, keyAgreement, keyCertSign, cRLSign, encipherOnly, decipherOnly

→ **Subject Alternative Name**

- ⇒ The subject alternative name extension allows identities to be bound to the subject of the certificate. These identities may be included in addition to or in place of the identity in the subject field of the certificate

→ **Extended Key Usage**

- ⇒ This extension indicates one or more purposes for which the certified public key may be used, in addition to or in place of the basic purposes indicated in the key usage extension.
- ⇒ TLS WWW server authentication, TLS WWW client authentication, Signing of downloadable executable code, Email protection, Timestamping

See <http://tools.ietf.org/html/rfc5280> for the complete list

Certificate revocation

→ **A PKI MUST include mechanisms for handling compromised certificates!**

⇒ Examples:

- Public keys whose private keys have been disclosed
- Public keys whose private keys have been lost
- Public keys not anymore used

→ **Two major (and coexisting) approaches**

- ⇒ Validity period for a certificate
- ⇒ Explicit revocation

→ **Classical analogy: a credit card**

Revocation approach: *Certificate Revocation List (CRL)*

- ➔ **EVERY CA must regularly publish a list of certificates the CA has revoked**
- ➔ **List format: must include**
 - ⇒ Issuer, Last update date, Next update date
 - ⇒ List of serial numbers revoked, along with revocation date
 - ⇒ CA digital signature
- ➔ **In principle, you SHOULD verify that the certificate of the site you are accessing has NOT been revoked**
 - ⇒ Overhead:
 - Online access, or offline periodic download of CRL
 - ⇒ In practice... frequently overlooked by non critical applications
 - But remember, whenever you are involved in a security critical application!